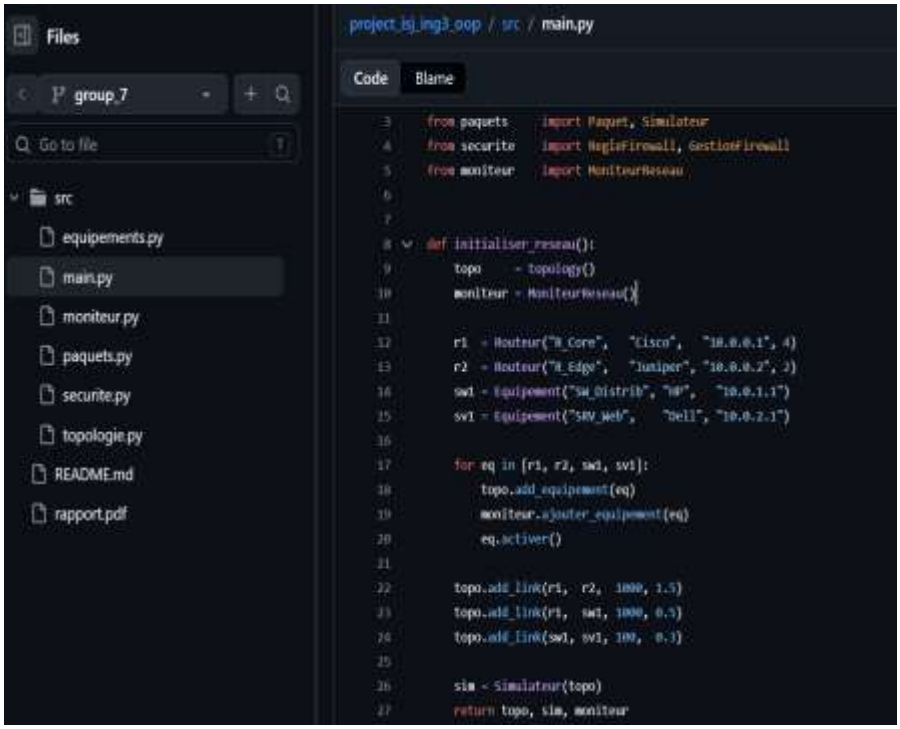


SIMNET – SMART NETWORK SIMULATOR IN PYTHON

GROUP 7

GROUP MEMEBERS
-EKANJE VANESSA EWOSE
-ALOBWEDE LESNAR AHONE
-BAYI NGANTCHEU ANGE FRANKA
-TSOFACK NGUE SOREL YVANA

Supervised by : M. Stephane Fedim

COURSE TITLE: Programmation Orientée Objet en Python

TABLE OF CONTENTS

1. INTRODUCTION

This project presents the design and implementation of SIMNet, a Smart Network Simulator developed in Python using object-oriented programming principles. The simulator models network devices such as routers, switches, firewalls, servers, and terminals while simulating packet transmission and network monitoring.this project enables us to understand communication in a network and applies OOP concepts in a practical case.

2. PROJECT OBJECTIVES

2.1 General Objective:

To design and implement a smart network simulator using Python and OOP concepts.

2.2 Specific Objectives:

- Model network devices
- Simulate packet routing
- Implement topology management
- Apply firewall filtering rules
- Monitor network traffic
- Generate reports
- Provide interactive console management

3. GROUP PRESENTATION AND ROLE DISTRIBUTION

Member	Role
EKANJE VANESSA EWOSE	equipment.py
ALOBWEDE LESNAR AHONE	topology.py
BAYI NGANTCHEU ANGE FRANKA	packet.py + main.py
TSOFACK NGUE SOREL YVANA	security.py + monitor.py + main.py

3.2 WORK METHODOLOGY

The project was developed by the collective efforts of each and every member above.this was possible by collaboratively using modular programming and github.each member was responsible for a specific file and module while mainteaning integration compactibility with each and every co-member.this was ensured by regular testing and git commits and modification of the overall system.

4. SYSTEM ANALYSIS

4.1 FUNCTIONAL DESCRIPTION and main features

SIMNet simulates the behavior of a real life computer network environment by:

- Allowing users to add devices and create links between multiple devices
- Allows packet transmission between devices
- Allows firewall filtering of packets
- Monitors traffic and activities
- Generate reports
- Interactive menu

5. OBJECT-ORIENTED DESIGN

The project applies the four major OOP concepts:

Encapsulation:

Data and behaviors are grouped inside classes.EXAMPLE the attribute address_ip is protected by a setter which validates the format of IPv4

```
18          @property
19          def adresse_ip(self):
20              return self._adresse_ip
21
22          @adresse_ip.setter
23          def adresse_ip(self, valeur):
24              """Valide le format IPv4 avant d'affecter."""
25              parties = valeur.split(".")
26              if len(parties) != 4:
27                  raise ValueError(f"Adresse IP invalide : {va
28              for partie in parties:
29                  if not partie.isdigit() or not (0 <= int(pa
30                      raise ValueError(f"Adresse IP invalide
31              self._adresse_ip = valeur
32
```

Inheritance:

All network devices inherit from a common Equipement base class.example: router inherits equipment using super().__init__()

```
72  class Routeur(Equipement):
73      """Modélise un routeur réseau avec table de routage
74
75      def __init__(self, nom, marque, adresse_ip, nb_inter
76          super().__init__(nom, marque, adresse_ip)
77          self.nb_interfaces = nb_interfaces
78          self.table_routage = []
```

Abstraction:

Generic device behaviors are defined through abstract classes and methods.

Polymorphism:

Different devices implement packet handling behaviors differently.example: the network monitor centralizes the information on equipements and traffic using the method diagnostiquer() of the class Equipement

```
51      def diagnostiquer(self):
52          """Retourne un dictionnaire d'informations pour
53          return {
54              "type": self.__class__.__name__,
55              "nom": self.nom,
56              "ip": self._adresse_ip,
57              "marque": self.marque,
58              "statut": "ACTIF" if self.est_actif else "I
59          }
```

Class method

Works on the class itself and is shared by all instances. Example

```
        Paquet._nb_paquets_crees += 1
        self.id = Paquet._nb_paquets_crees

    @classmethod
    def get_nb_paquets_crees(cls):
        """Retourne le nombre total de paquets créés."""
        return cls._nb_paquets_crees

    def __str__(self):
```

5.2 Justification of Design Choices

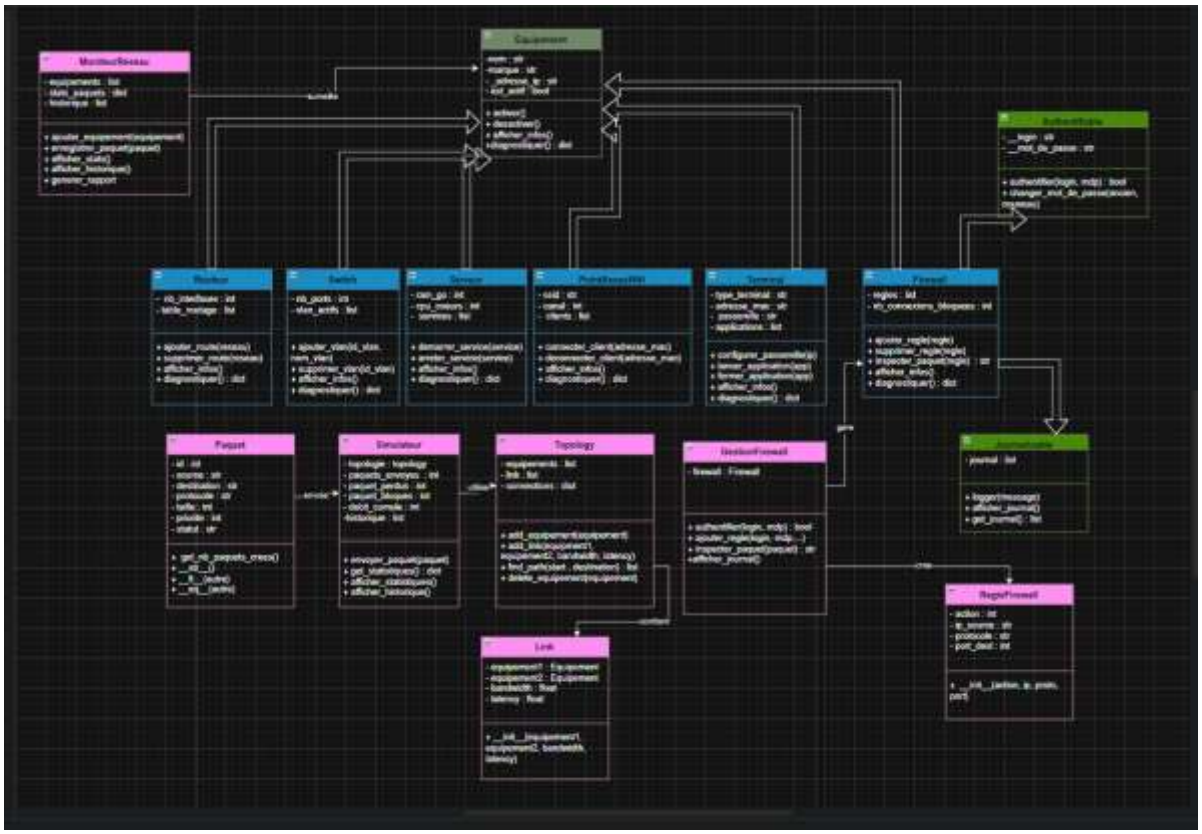
I. use of BFS:

The topology module uses an adjacency-list graph structure because it allows efficient storage and traversal of network connections.

II. USE OF adjacency dictionary

The topology module uses an adjacency-list graph structure because it allows efficient storage and traversal of network connections.

6. CLASS DIAGRAM



7. Module Implementation

equipment.py:

Implements the equipment hierarchy and device behaviors.

topology.py:

Manages devices, links, graph structures, and pathfinding preparation.

packet.py:

Handles packet transmission and routing logic.

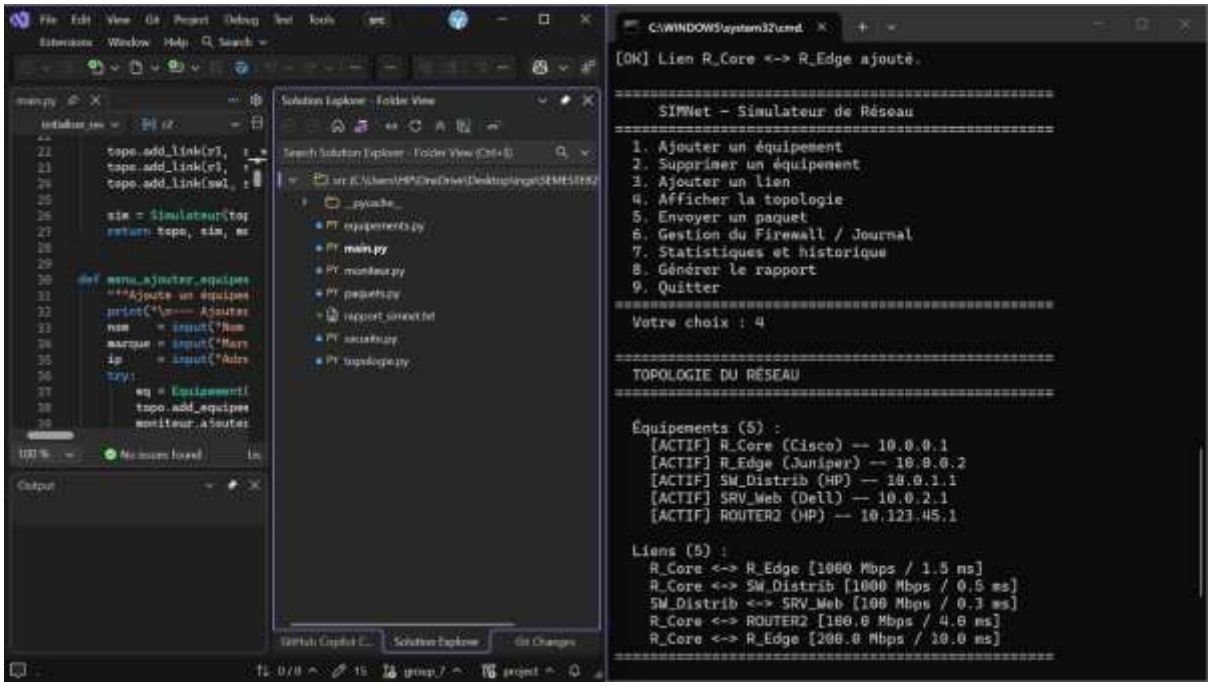
security.py:

Implements firewall filtering and packet security.

monitor.py and main.py:

Provide monitoring features and user interaction.

8. IMPLEMENTATION EXAMPLES



9. DIFFICULTIES ENCOUNTERED

Several challenges were encountered during development:
- Understanding OOP relationships
- Implementing graph structures
- Integrating modules
- Managing GitHub collaboration
- Debugging pathfinding algorithms

10. Solutions Provided

To solve these challenges:
- BFS was used for pathfinding
- Naming conventions were standardized
- Modules were separated clearly
- Regular testing was performed
- GitHub commits improved collaboration

12. FUTURE IMPROVEMENTS

Future versions of the simulator may include:
- Graphical user interface (GUI)
- Dynamic routing protocols
- VLAN simulation
- Real-time packet analysis
- Database integration

13. CONCLUSION

This project allowed the team to apply object-oriented programming concepts in a practical networking context. The development of SIMNet improved our understanding of teamwork, debugging, GitHub, OOP Concepts and network simulation.

14. REFERENCES

- Python Official Documentation: https://docs.python.org
- GitHub Documentation: https://docs.github.com
- OOP Course Notes
- Networking Fundamentals Documentation

15. Appendices

Screenshot of git commit